

作业 2

一. 阅读理解（共3题，60分）

1. (阅读理解) C语言中允许无符号数和有符号整数之间的转换，下面是一段C语言代码：

```
int x = -1;
unsigned u = 2147483648;
printf("x=%u=%d\n", x, x);
printf("u=%u=%d\n", u, u);
```

(1) (简答题)

给出在32位计算机中上述程序段的输出结果。

答：

在32位计算机中，输出结果如下：

```
x=4294967295=-1
```

```
u=2147483648=-2147483648
```

(2) (简答题)

分析上述结果出现的原因。

答：

- 对于 `int x = -1;` :
 - 在32位系统中，`int` 类型的 `-1` 在内存中以补码形式存储为 `0xFFFFFFFF`（即二进制的32个1）。
 - 当使用 `%u` 格式符打印时，该 `0xFFFFFFFF` 被解释为无符号整数，其值为 $2^{32} - 1 = 4294967295$ 。
 - 当使用 `%d` 格式符打印时，该 `0xFFFFFFFF` 被正确解释为有符号整数，其值为 `-1`。
- 对于 `unsigned u = 2147483648;` :
 - `unsigned int` 类型的 `2147483648` 在内存中存储为 `0x80000000`（即二进制的 `1000 ... 000`，最高位为1）。
 - 当使用 `%u` 格式符打印时，该 `0x80000000` 被正确解释为无符号整数，其值为 `2147483648`。

- 当使用 `%d` 格式符打印时, 该 `0x80000000` 被解释为有符号整数。在补码表示中, 最高位为1表示负数。在32位有符号整数中, `0x80000000` 代表最小的负数, 即 $-2^{31} = -2147483648$ 。这是因为 `2147483648` 大于32位有符号整数的最大正值 ($2^{31} - 1 = 2147483647$), 因此在按有符号数解释时发生了溢出。

2. (阅读理解) 将下列十进制数表示成浮点规格化数, 阶码4位, 尾数10位, 各含1位符号, 阶码和尾数均用补码表示。

(1) (填空题)

57/128的4位阶码 = (**1111**), 10位尾数 = (**0111001000**)。

解答过程:

- 转换为二进制小数: $57/128 = 0.4453125 = 0.0111001_2$
- 规格化: $0.0111001_2 \times 2^{-1}$
- 阶码 (Exponent): $E = -1$ 。4位补码表示 -1 为 **1111**。
- 尾数 (Mantissa): 数值为 0.0111001_2 。由于是正数, 符号位为 **0**。10位尾数 (包含1位符号位和9位数值位) 表示为 **0.111001000**。

(2) (填空题)

-69/128的4位阶码 = (**0000**), 10位尾数 = (**1.011101100**)。

解答过程:

- 转换为二进制小数的绝对值: $69/128 = 0.5390625 = 0.1000101_2$
- 规格化: $N = -0.1000101_2 \times 2^0$
- 阶码 (Exponent): $E = 0$ 。4位补码表示 0 为 **0000**。
- 尾数 (Mantissa): 数值为 -0.1000101_2 。由于是负数, 符号位为 **1**。
 - 正数 0.100010100 (10位, 补齐9位小数)
 - 按位取反 **1.011101011**
 - 末位加1 **1.011101100**
 - 所以10位尾数 (补码) 为 **1.011101100**。

3. (阅读理解) 设二进制浮点数的阶码为3位, 尾数是7位。用模2补码写出它们所能表示的最大正数、最小正数、最大负数和最小负数。

您好！我将基于浮点数格式（阶码 E 占 3 位，尾数 M 占 7 位，都用模 2 补码表示）来帮您计算它们所能表示的最大正数、最小正数、最大负数和最小负数。

浮点数的表示通常为：

$$N = (-1)^S \times M \times 2^E$$

但在计算机中，当阶码 E 和尾数 M 都采用**模 2 补码**表示时：

- **符号位**：补码表示的最高位即为符号位（0 为正，1 为负）。
- **规格化**：为了保证尾数的精度，通常要求尾数**规格化**。

1. 尾数 M (7 位补码)

尾数 M 占 7 位 ($M = S_M.m_1m_2\dots m_6$)，通常要求规格化，即 $|M| \geq 1/2$ 。

- **最大正尾数 M_{max}^+** ：
 - 符号位 $S_M = 0$ 。其余位全为 1。
 - $M_{max}^+ = 0.111111_2 = 1 - 2^{-6} = 63/64$ 。
- **最小正尾数 M_{min}^+** ：
 - 符号位 $S_M = 0$ 。为了规格化，第一位小数 m_1 必须为 1，其余为 0。
 - $M_{min}^+ = 0.100000_2 = 1/2$ 。
- **最大负尾数 M_{max}^-** ：
 - M 的绝对值最小，符号位为 1。**注意**：负数规格化时，一般要求 $1.000000_2 \leq M < 1.100000_2$ 。在补码中，绝对值最小的负数是符号位为 1，其余位全为 0。
 - $M_{max}^- = 1.000000_2 = -1$ 。
- **最小负尾数 M_{min}^-** ：
 - M 的绝对值最大。在补码中，绝对值最大的负数是 1.000001_2 。
 - $M_{min}^- = 1.000001_2 = -1 + 2^{-6} = -63/64$ 。

2. 阶码 E (3 位补码)

阶码 E 占 3 位 ($E = S_E e_1 e_2$)，表示范围如下：

补码表示 (3位)	实际值 E
011	+3 (最大正阶码 E_{max}^+)
010	+2
001	+1
000	0
111	-1
110	-2
101	-3
100	-4 (最小负阶码 E_{min}^-)

1234

所能表示的极值

1. 最大正数 N_{max}^+

最大正数由最大正尾数和最大正阶码组合而成：

$$N_{max}^+ = M_{max}^+ \times 2^{E_{max}^+}$$

- $M_{max}^+ = 0.111111_2 = 63/64$
- $E_{max}^+ = 011_2 = +3$

$$N_{max}^+ = (1 - 2^{-6}) \times 2^3 = 8 - 2^{-3} = 7.875$$

模 2 补码表示：

- 尾数: 0111111
- 阶码: 011
- 结果: 011 0111111

2. 最小正数 N_{min}^+

最小正数由最小正尾数和最小负阶码组合而成：

$$N_{min}^+ = M_{min}^+ \times 2^{E_{min}^-}$$

- $M_{min}^+ = 0.100000_2 = 1/2$
- $E_{min}^- = 100_2 = -4$

$$N_{min}^+ = (1/2) \times 2^{-4} = 2^{-5} = 1/32 = 0.03125$$

模 2 补码表示：

- 尾数: 0100000
- 阶码: 100
- 结果: 100 0100000

3. 最小负数 N_{min}^- (绝对值最大)

最小负数（绝对值最大的负数）由绝对值最大的负尾数和最大正阶码组合而成：

$$N_{min}^- = M_{min}^- \times 2^{E_{max}^+}$$

- $M_{min}^- = 1.000001_2 = -63/64$
- $E_{max}^+ = 011_2 = +3$

$$N_{min}^- = (-1 + 2^{-6}) \times 2^3 = -8 + 2^{-3} = -7.875$$

模 2 补码表示：

- 尾数: 1000001
- 阶码: 011
- 结果: 011 1000001

4. 最大负数 N_{max}^- (绝对值最小)

最大负数（绝对值最小的负数）由绝对值最小的负尾数和最小负阶码组合而成：

$$N_{max}^- = M_{max}^- \times 2^{E_{min}^-}$$

- $M_{max}^- = 1.000000_2 = -1$
- $E_{min}^- = 100_2 = -4$

$$N_{max}^- = (-1) \times 2^{-4} = -1/16 = -0.0625$$

模 2 补码表示：

- 尾数: 1000000
- 阶码: 100
- 结果: 100 1000000

总结

以下是您所求的四个极值的补码表示和对应的十进制值：

极值类型	尾数 M (7位补码)	阶码 E (3位补码)	补码表示 (E M)	十进制值
最大正数 N_{max}^+	0111111	011	011 0111111	7.875

极值类型	尾数 M (7位补码)	阶码 E (3位补码)	补码表示 (E M)	十进制值
最小正数 N_{min}^+	0100000	100	100 0100000	0.03125
最小负数 N_{min}^-	1000001	011	011 1000001	-7.875
最大负数 N_{max}^-	1000000	100	100 1000000	-0.0625

阶码 (3位补码): 可表示的范围是 100 (-4) 到 011 (3)。

尾数 (7位补码, 且为规格化小数形式):

- 最大正尾数: 0.111111 (约等于 $1 - 2^{-6}$)
- 最小正尾数: 0.100000 (0.5)
- 最大负尾数 (最接近0的负数): 1.111111 (-2^{-6})
- 最小负尾数 (离0最远的负数): 1.000000 (-1)

(1) (填空题)

最大正数的3位阶码为 (011), 7位尾数为 (0.111111)。

解释: 为了得到最大正数, 需取最大的正尾数 0.111111 和最大的正阶码 011 (十进制3)。

(2) (填空题)

最小正数的3位阶码为 (100), 7位尾数为 (0.100000)。

解释: 为了得到最小正数, 需取最小的规格化正尾数 0.100000 和最小的阶码 100 (十进制-4)。

(3) (填空题)

最大负数的3位阶码为 (100), 7位尾数为 (1.111111)。

解释: 为了得到最大负数 (最接近0的负数), 需取最接近0的负尾数 1.111111 (十进制 -2^{-6}) 和最小的阶码 100 (十进制-4)。

(4) (填空题)

最小负数的3位阶码为 (011), 7位尾数为 (1.000000)。

解释： 为了得到最小负数 (离0最远的负数)，需取离0最远的负尾数 1.000000 (十进制 -1) 和最大的阶码 011 (十进制3)。

二. 简答题 (共1题, 20分)

4. (简答题)

分析下列几种情况下所能表示的数据范围分别是多少？

我们将 $N = 16$ 位数据分为符号位 S (最高位) 和数值位。

1. 16 位无符号数 (Unsigned Integer)

无符号数的所有 16 位都用于表示数值的大小。

计算过程：

1. 最小值： 当所有位都为 0 时，数值最小。

$$\underbrace{0000000000000000}_{16 \text{ 位}} = 0$$

2. 最大值： 当所有位都为 1 时，数值最大。

$$\underbrace{1111111111111111}_{16 \text{ 位}} = \sum_{i=0}^{N-1} 2^i = 2^N - 1$$

代入 $N = 16$ ：

$$2^{16} - 1 = 65536 - 1 = 65535$$

数据范围： $[0, 2^{16} - 1] = [0, 65535]$

2. 16 位原码定点小数 (Signed Magnitude Fixed-Point Fraction)

原码定点小数的格式为 $S.m_1m_2\dots m_{15}$ ，其中 S 是符号位， $m_1\dots m_{15}$ 是 15 位数值位。

计算过程：

1. 最大正数 X_{max}^+ ：

- 符号位 $S = 0$ 。
- 其余 15 位数值位全为 1 (表示最大小数 0.999...)。

原码 : 0.111111111111111_2

$$\text{数值} = +(2^{-1} + 2^{-2} + \dots + 2^{-15}) \quad \text{数值} = +(\sum_{i=1}^{15} 2^{-i}) = +(1 - 2^{-15})$$

2. 最小负数 X_{min}^- ：

- 符号位 $S = 1$ (负号)。
- 其余 15 位数值位全为 1 (表示绝对值最大)。

原码 : 1.111111111111111_2

$$\text{数值} = -(2^{-1} + 2^{-2} + \dots + 2^{-15}) = -(1 - 2^{-15})$$

3. 最小正数和最大负数： 分别是 $+0$ (0.00...0) 和 -0 (1.00...0)。

数据范围： $[-(1 - 2^{-15}), +(1 - 2^{-15})]$

3. 16 位补码定点小数 (Two's Complement Fixed-Point Fraction)

补码定点小数的格式为 $S.m_1m_2\dots m_{15}$ ，其数值 X 由以下公式计算：

$$X = -S \cdot 2^0 + \sum_{i=1}^{15} m_i \cdot 2^{-i}$$

计算过程：

1. 最大正数 X_{max}^+ ：

- 符号位 $S = 0$ ，其余数值位全为 1。

补码：0.111111111111111₂

$$\text{数值} = 0 + (2^{-1} + 2^{-2} + \dots + 2^{-15}) = 1 - 2^{-15}$$

2. 最小负数 X_{min}^- ：

- 补码中绝对值最大的负数，由 1 后面跟所有 0 构成。

补码：1.000000000000000₂

$$\text{数值} = -1 \cdot 2^0 + 0 = -1$$

3. 最大负数：补码 1.111...1₂ (即 1 的真值补码) 代表 -2^{-15} 。

数据范围： $[-1, 1 - 2^{-15}]$

4. 16 位补码定点整数 (Two's Complement Fixed-Point Integer)

补码定点整数的格式为 $Sx_{14}x_{13}\dots x_0$ ，其中 S 是符号位，其余 15 位是数值位。其数值 X 由以下公式计算：

$$X = -S \cdot 2^{15} + \sum_{i=0}^{14} x_i \cdot 2^i$$

计算过程：

1. 最大正数 X_{max}^+ ：

- 符号位 $S = 0$ ，其余 15 位数值位全为 1。

补码：011111111111111₂

$$\text{数值} = 0 + (2^{14} + 2^{13} + \dots + 2^0) = 2^{15} - 1 = 32767$$

2. 最小负数 X_{min}^- :

- 补码中绝对值最大的负数，由 1 后面跟所有 0 构成。

补码: 1000000000000000_2

$$\text{数值} = -1 \cdot 2^{15} + 0 = -2^{15} = -32768 \quad \text{【关键点】: 补码可以多表示一个负数。}$$

数据范围: $[-2^{15}, 2^{15} - 1] = [-32768, 32767]$

1) 16位无符号数;

答: 0 到 $2^{16} - 1$, 即 0 到 65535。

2) 16位原码定点小数;

答: 1位符号位, 15位小数位。

- 最大正数: 0.11111111111111_2 ($1 - 2^{-15}$)
- 最小负数: 1.11111111111111_2 ($-(1 - 2^{-15})$)
- 范围: $-(1 - 2^{-15})$ 到 $(1 - 2^{-15})$ (包含 0)。

3) 16位补码定点小数;

答: 1位符号位, 15位小数位。

- 最大正数: 0.11111111111111_2 ($1 - 2^{-15}$)
- 最小负数: 1.00000000000000_2 (-1)
- 范围: -1 到 $(1 - 2^{-15})$ 。

4) 16位补码定点整数。

答: 1位符号位, 15位数值位。

- 最小负数: 100000000000000_2 ($-2^{15} = -32768$)
- 最大正数: 011111111111111_2 ($2^{15} - 1 = 32767$)

- 范围： -32768 到 32767 。

三. 填空题（共1题，20分）

5. (填空题) 与单精度浮点数43940000H对应的十进制数是（ 296.0 ）。

解答过程：

- 单精度浮点数采用IEEE 754标准，32位。
- 43940000H 转换为二进制： 0100 0011 1001 0100 0000 0000 0000 0000
- 符号位 (S)： 第31位，为 0 (表示正数)。
- 阶码 (E)： 第30-23位，为 10000111。
 - 转换为十进制： $128 + 4 + 2 + 1 = 135$ 。
 - 根据IEEE 754标准，单精度浮点数的阶码需要减去一个偏移量 (Bias)，Bias = 127 。
 - 实际阶码 = $135 - 127 = 8$ 。
- 尾数 (M)： 第22-0位，为 001010000000000000000000。
 - 由于是规格化浮点数，尾数部分需要加上一个隐藏的 1 。
 - 有效位为 1.00101 (后续的0不影响数值)。
 - 转换为十进制： $1 + 0*(2^{-1}) + 0*(2^{-2}) + 1*(2^{-3}) + 0*(2^{-4}) + 1*(2^{-5})$
 - $1 + 0 + 0 + 0.125 + 0 + 0.03125 = 1.15625$ 。
- 最终结果： $(-1)^S \times 1.M \times 2^{\text{实际阶码}}$
 - $(-1)^0 \times 1.15625 \times 2^8$
 - $1 \times 1.15625 \times 256$
 - $1.15625 \times 256 = 296.0$ 。